BOOK A DOCTOR SYSTEM

Frontend Development and Explanation - React.js, Routing, Axios and Role-Based Dashboards

This document explains the frontend implementation of the Book a Doctor MERN application. The client is built using React.js and Vite. React Router manages page navigation, Axios communicates with the Express backend, localStorage stores the JWT token, and CSS provides responsive styling for Admin, Doctor, and Patient modules.

1. main.jsx - React Application Entry Point

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";

import App from "./App.jsx";
import "./index.css";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

• Creates the React root and renders the application.

• Wraps App with BrowserRouter for client-side navigation.

• Loads the global stylesheet.

2. App.jsx - Role-Based Routes

```
import { Routes, Route } from "react-router-dom";

import Home from "./pages/Home";
import Login from "./pages/Login";
import Register from "./pages/Register";

import AdminDashboard from "./pages/AdminDashboard";
import DoctorDashboard from "./pages/DoctorDashboard";
import PatientDashboard from "./pages/PatientDashboard";

import Doctors from "./pages/Doctors";
import Patients from "./pages/Patients";
import Appointments from "./pages/Appointments";
import PatientDoctors from "./pages/PatientDoctors";
import BookAppointment from "./pages/BookAppointment";
import MyAppointments from "./pages/MyAppointments";
import DoctorAppointments from "./pages/DoctorAppointments";
import Profile from "./pages/Profile";

function App() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/login" element={<Login />} />
      <Route path="/register" element={<Register />} />

      <Route path="/admin" element={<AdminDashboard />} />
      <Route path="/admin/doctors" element={<Doctors />} />
      <Route path="/admin/patients" element={<Patients />} />
      <Route path="/admin/appointments" element={<Appointments />} />

      <Route path="/patient" element={<PatientDashboard />} />
      <Route path="/patient/doctors" element={<PatientDoctors />} />
      <Route path="/patient/book/:id" element={<BookAppointment />} />
```

```
      <Route path="/patient/appointments" element={<MyAppointments />} />

      <Route path="/doctor" element={<DoctorDashboard />} />
      <Route
        path="/doctor/appointments"
        element={<DoctorAppointments />}
      />

      <Route path="/profile" element={<Profile />} />
    </Routes>
  );
}

export default App;
```

• Defines public, Admin, Doctor, Patient, and Profile routes.

• Uses a dynamic doctor id for the booking page.

• Keeps all page navigation in one central component.

3. services/api.js - Axios and JWT Integration

```
import axios from "axios";

const api = axios.create({
  baseURL: "http://localhost:5000/api",
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");

  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }

  return config;
});

export default api;
```

• Creates one reusable Axios client.

• Automatically attaches the JWT token to protected requests.

• Connects the React client to the Express API.

4. Login.jsx - Authentication and Redirection

```
const login = async () => {
  try {
    const res = await api.post("/auth/login", {
      email,
      password,
    });

    localStorage.setItem(
      "token",
      res.data.data.token
    );

    const role = res.data.data.user.role;

    alert("Login Successful");

    if (role === "admin") navigate("/admin");
    else if (role === "doctor") navigate("/doctor");
    else navigate("/patient");
  } catch (err) {
    alert(
      err.response?.data?.message ||
      err.message
    );
  }
};
```

• Sends email and password to the backend login API.

• Stores the JWT token after successful authentication.

• Redirects Admin, Doctor, and Patient users to different dashboards.

5. Register.jsx - Account Creation

```
const [form, setForm] = useState({
  firstName: "",
  lastName: "",
  email: "",
  password: "",
  phone: "",
  role: "patient",
});

const handleChange = (e) => {
  setForm({
    ...form,
    [e.target.name]: e.target.value,
  });
};

const register = async (e) => {
  e.preventDefault();

  try {
    await api.post("/auth/register", form);

    alert("Registration Successful");
    navigate("/login");
  } catch (error) {
    alert(
      error.response?.data?.message ||
      error.message
    );
  }
};
```

• Uses one state object to manage the complete form.

• Calls the registration API and redirects to Login.

• Supports Patient and Doctor role selection.

6. PatientDashboard.jsx - Patient Module Entry

```
function PatientDashboard() {
  return (
    <div className="patient-dashboard">
      <div className="patient-header">
        <h1>Patient Dashboard</h1>
        <p>
          Welcome to Online Doctor
          Appointment System
        </p>
      </div>

      <div className="patient-cards">
        <Link to="/patient/doctors" className="patient-card">
          <FaUserMd className="card-icon" />
          <h2>Doctors</h2>
          <p>Browse available doctors</p>
        </Link>

        <Link
          to="/patient/appointments"
          className="patient-card"
        >
          <FaCalendarCheck className="card-icon" />
          <h2>Appointments</h2>
          <p>View your appointments</p>
        </Link>

        <Link to="/profile" className="patient-card">
          <FaUser className="card-icon" />
          <h2>Profile</h2>
          <p>Manage your account</p>
        </Link>
      </div>
    </div>
```

```
  );
}
```

• Provides quick links to Doctors, Appointments, and Profile.

• Uses React Router Link and React Icons.

• Acts as the main entry page for a patient.

7. PatientDoctors.jsx - Fetch and Display Doctors

```
useEffect(() => {
  fetchDoctors();
}, []);

const fetchDoctors = async () => {
  try {
    const res = await api.get("/doctors");
    setDoctors(res.data.data || []);
  } catch (error) {
    alert(
      error.response?.data?.message ||
      error.message
    );
  } finally {
    setLoading(false);
  }
};

const bookDoctor = (doctorId) => {
  navigate(`/patient/book/${doctorId}`);
};
```

• Loads doctor records when the page opens.

• Displays specialization, hospital, experience, and fee.

• Navigates to the booking form using the selected doctor id.

8. BookAppointment.jsx - Appointment Creation

```
const { id } = useParams();

const submitAppointment = async (e) => {
  e.preventDefault();

  try {
    await api.post("/appointments", {
      doctorId: id,
      appointmentDate,
      appointmentTime,
      reason,
    });

    alert("Appointment booked successfully");
    navigate("/patient/appointments");
  } catch (error) {
    alert(
      error.response?.data?.message ||
      error.message
    );
  }
};
```

• Reads the selected doctor id from the URL.

• Sends date, time, and reason to the appointment API.

• Redirects to appointment history after booking.

9. MyAppointments.jsx - Patient Appointment History

```
useEffect(() => {
  fetchAppointments();
}, []);

const fetchAppointments = async () => {
  try {
    const res = await api.get(
      "/appointments/my"
    );

    setAppointments(
      res.data.data || []
    );
  } catch (error) {
    alert(
      error.response?.data?.message ||
      error.message
    );
  } finally {
    setLoading(false);
  }
};
```

• Fetches only the logged-in patient's appointments.

• Shows doctor details, date, time, and appointment status.

• Handles loading and empty appointment states.

10. DoctorAppointments.jsx - Doctor Status Management

```
const fetchAppointments = async () => {
  const res = await api.get(
    "/appointments/doctor"
  );

  setAppointments(
    res.data.data || []
  );
};

const updateStatus = async (
  appointmentId,
  status
) => {
  try {
    await api.put(
      `/appointments/${appointmentId}/status`,
      { status }
    );

    fetchAppointments();
  } catch (error) {
    alert(
      error.response?.data?.message ||
      error.message
    );
  }
};
```

• Loads appointments assigned to the logged-in doctor.

• Updates status to Confirmed, Completed, or Cancelled.

• Refreshes the list after each change.

11. AdminDashboard.jsx - Dashboard Statistics

```
const [stats, setStats] = useState({
  totalUsers: 0,
  doctors: 0,
  patients: 0,
  appointments: 0,
  pending: 0,
  confirmed: 0,
});

useEffect(() => {
  fetchStats();
}, []);

const fetchStats = async () => {
  try {
    const res = await api.get(
      "/admin/dashboard"
    );

    setStats(res.data.data);
  } catch (error) {
    alert(
      error.response?.data?.message ||
      error.message
    );
  }
};
```

• Loads live system totals from the protected admin API.

• Uses reusable cards to show users, doctors, patients, and appointments.

• Keeps the dashboard synchronized with MongoDB data.

12. Admin Management Pages - CRUD Operations

```
// Read doctors
const res = await api.get("/doctors");

// Create doctor
await api.post("/doctors", doctorForm);

// Update doctor
await api.put(
  `/doctors/${doctorId}`,
  doctorForm
);

// Delete doctor
await api.delete(
  `/doctors/${doctorId}`
);

// Read users
const users = await api.get("/admin/users");

// Read appointments
const appointments = await api.get(
  "/admin/appointments"
);

// Update appointment status
await api.put(
  `/admin/appointments/${id}/status`,
  { status }
);
```

• Implements Create, Read, Update, and Delete operations for doctor management.

• Allows administrators to view users and appointments.

• Uses protected APIs for admin appointment status updates.

13. Profile.jsx - Profile Retrieval and Update

```
useEffect(() => {
  loadProfile();
}, []);

const loadProfile = async () => {
  const res = await api.get("/auth/me");
  setProfile(res.data.data);
};

const updateProfile = async (e) => {
  e.preventDefault();

  try {
    const res = await api.put(
      "/auth/profile",
      {
        firstName: profile.firstName,
        lastName: profile.lastName,
        phone: profile.phone,
      }
    );

    setProfile(res.data.data);
    alert("Profile updated");
  } catch (error) {
    alert(
      error.response?.data?.message ||
      error.message
    );
  }
};
```

• Retrieves profile data through a protected endpoint.

• Updates first name, last name, and phone.

• Keeps the email field protected as the account identifier.

14. Reusable Components and CSS

```
src/components/
  DashboardCard.jsx
  DoctorCard.jsx
  Footer.jsx
  Navbar.jsx
  Sidebar.jsx
  Topbar.jsx

src/styles/
  Dashboard.css
  DoctorCard.css
  Login.css
  Register.css
  PatientDashboard.css
  PatientDoctors.css
  BookAppointment.css
  MyAppointments.css
  DoctorAppointments.css
  Profile.css
```

• Reusable components reduce repeated JSX.

• Each major page has a dedicated CSS file.

• The layout supports cards, tables, forms, navigation, and responsive dashboards.

15. Frontend Module Summary

Module	Pages / Components	Purpose
Public	Home, Login, Register	Landing page, authentication, and account creation.
Patient	PatientDashboard, PatientDoctors, BookAppointment, MyAppointments	Browse doctors, book appointments, and track status.
Doctor	DoctorDashboard, DoctorAppointments	Review assigned appointments and update status.
Admin	AdminDashboard, Doctors, Patients, Appointments	Manage platform records and dashboard statistics.
Shared	Navbar, Sidebar, Topbar, Footer, DashboardCard, DoctorCard	Reusable UI building blocks.
Integration	api.js and AuthContext.jsx	Backend communication and shared authentication state.

16. Complete Frontend Request Flow

User Action -> React Component -> Event Handler -> Axios Request -> Express Backend -> JSON Response -> React State Update -> Updated UI

Conclusion

The Book a Doctor frontend uses React components, role-based routing, controlled forms, Axios API integration, JWT token handling, reusable UI components, and dedicated CSS. The application provides complete interfaces for Patient, Doctor, and Admin workflows and communicates securely with the Node.js and Express backend.